Project in Machine Learning

Supervised learning
using numerical and labeled data from unknown distributions

Edvard Edsgård

June, 2026 - Gothenburg

1 Construct a ML pipeline

1.1 Exploration of the given dataset

The data and scoring metrics:
For this problem, the given dataset was called `X_TR`, with labels `y_TR`. Consisting of 8080 samples, each having 820 features. There was seven different class labels, distributed as:

- label 1 - 1309 samples
- label 2 - 1071 samples
- label 3 - 1211 samples
- label 4 - 1568 samples
- label 5 - 1202 samples
- label 6 - 1227 samples
- label 7 - 492 samples

It can be seen that class imbalance exist. Especially for class 7. There are no missing values in the data set.

When training models on imbalanced, multiclass data, standard accuracy can be highly misleading. Instead, macro-averaged f1-score, Cohen's Kappa, and balanced accuracy are more appropriate metrics to use. These metrics account for the unequal distribution of classes without letting majority classes skew the evaluation.

I chose `f1_macro` as the primary scoring metric used for comparing model performance, as well as for conducting hyperparameter tuning, since the *Macro-Averaged f1-score* calculates the f1-score for each class independently and takes their unweighted average. This is good when minority classes are just as important as majority classes. I assume that all classes are equally important, since I do not have any information indicating something else. To further tackle the imbalance of the data, I decided to use the hyperparameter choice `class_weight='balanced'` for all classifiers where this option was available. For the models to be evaluated in this project, `class_weight='balanced'` was used with *Logistic Regression*, *Random Forest*, *Linear SVC*, and *SVC with polynomial kernel*. This option was not available for *KNN*.

The "balanced" mode uses the value counts of each label to automatically adjust weights inversely proportional to class frequencies in the input data. This puts greater importance on each sample of a minority class.

I decided that these decisions would be appropriate to tackle the class imbalance of the training data, and therefore, I did **not** conduct any up-sampling or down-sampling of the data.

Data Exploration:
I extracted ≈ 3000 samples that was used for data exploration. It was used to conduct a brief analysis, in order to determine which filtering methods to apply, `f_classif` or `mutual_info_classif`, and the level of filtering, i.e. how many features that are reasonable to keep. Further, this exploration data was used to evaluate if embedded feature selection using *Logistic regression with L1-penalty* as the estimator would produce increased performance. It was also evaluated if *Principal Component Analysis* (PCA) using different numbers of principal components affected the performance.

To measure performance on the data after feature selection and PCA, the classifiers *Logistic Regression with L2-penalty* and *Random Forest* were used. Performance was measured as the mean value from cross-validation `f1_macro` scores. Performance was compared between different combinations of filter techniques, levels of filtering, and embedded feature selection or PCA. Results can be seen in Figure 1.

I am aware that this approach gives rise to some data leakage, as some of the training data was seen and utilized to compare performance of combinations of feature selection and PCA. However, it

was deemed necessary to be able to make more targeted choices of feature selection methods and range of hyperparameters to be further evaluated, in order to decrease the computational burden.

The data exploration gave indications of equal performance of filtering methods ANOVA F-value and Mutual Information. I decided to proceed with ANOVA F-value as that method is faster.

Embedded feature selection using *Logistic Regression with L1-penalty* did not decease the feature space of pre-filtered input, using pre-filtered input in range $[10, 200]$. This approach was therefore deemed useless for this data set. Heatmaps of this can be seen in Appendix A.

To visualize the data, and the effect of PCA, I filtered the exploration data down to 20 features, using ANOVA F-test, and applied PCA. Scatterplots of PC1, PC2, PC3 and PC4 can be seen in Figure 2. In these dimensions of the feature space, all classes seem to be forming clusters, taking round and/or elliptical shapes. Looking at PC1 and PC2, if one were to remove class 5 (orange) and class 2 (blue), the remaining classes seem to be fairly well separated with slight overlaps on the edges of respective clusters. Looking at PC3 and PC4, if one were to remove all classes **except** class 5 (orange) and class 2 (blue), these two classes also seem to be fairly well separated. Altogether, this gives an indication that the data transformations applied may well be appropriate, as classes are clustered together, at least in dimensions PC1, PC2, PC3 and PC4.

Moving forward:
Observing the scores in heatmaps, Figure 1, and the PCA scatter plots, Figure 2, I decided to proceed with ANOVA F-value filtering, filtering levels, $k \in \{15, 20, 25, 30\}$, and PCA number of components, $n \in \{None, 12, 15\}$, where *None* means that PCA is applied, but the feature space is not decreased. These sets of hyperparameters were included in a Grid Search in combination with model specific hyperparameters, such as values of regularization parameter C, or number of neighbors for *KNN*.

1.2 Hyperparameter tuning and training

The below process was repeated three times. Each starting with a train-test-split of `X_TR` and `y_TR`, to extract 20% ($\approx$ 1600 samples) of the data to be used as a super-test, in order to be able to evaluate performance on unseen data at the very end of model training.

In the first run, none of the data points used for exploration were included in the super-test used to evaluate performance. For the second and third run, new random states were chosen and data was shuffled. Therefore, in the second and third run, data that was used for exploration can also have appeared in the super-test set.

In order to be able to evaluate the effect that different sample sizes will have on the results, I decided to conduct evaluation for sample sizes of 400, 1000, and 3750. These will be referred to as sample size "small", "medium", and "large". Using outer-outer fold $R = 5$, outer fold $K = 5$, and inner fold $L = 4$ (for hyperparameter tuning), this process provided 25 performance evaluations for each model and each sample size.

In each of the 25 folds:

- Sample size 400 $\rightarrow$ 320 samples for training, and 80 for testing
- Sample size 1000 $\rightarrow$ 800 samples for training and 200 for testing
- Sample size 3750 $\rightarrow$ 3000 samples for training and 750 for testing

After this procedure, the set of hyperparameters that was chosen most often by the Grid Search for each model were extracted, and fitted to new pipelines that included filtering, scaling, PCA, and the classifiers. These pipelines were fitted to all training data, and thereafter used to predict the labels from the super-test set. Predictions were then evaluated as confusion matrices, and as overall and class-specific performance scores.

1.3 Performance evaluation

Performance from the first run can be seen as Boxplots in Figure 3. Performance from the second and third run can be found in Appendix A. However, their appearance are very similar, indicating stable and consistent performance between runs.

Not surprisingly, best performance for all models were achieved using the "large" sample size for training. This makes sense since the models have more data to learn from. This can be seen from the higher placement of boxes trained on large sample size. The IQR of boxplots for "large" sample sizes are narrower. This indicates a more stable performance for the models trained on "large" sample size, compared to when the same models were trained on smaller sample sizes. But it could also be down to the fact that models trained on "large" sample size were evaluated on 750 samples per fold, compared to 200 for sample size "medium" and only 80 for sample size "small".

From the Boxplots it can be seen that *KNN* and *SVC with polynomial kernel* achieved the best scores, closely followed by *Random Forest*. *Logistic Regression* and *Linear SVC* performs slightly worse. However, it can be argued that all classifiers perform reasonably well, and all classifiers **except** *Linear SVC* consistently perform f1-macro, Kappa, and Accuracy scores above 0.9 when being trained on "large" sample sizes.

The same pattern emerge in Table 1. This table shows performance scores after pipelines has been re-fitted to all training data, and then been tasked to predict the super-test set, which constitutes of withheld data, not used for training nor collecting performance metrics seen in the Boxplots. In the Table, it can be seen that *KNN* and *SVC with polynomial kernel* perform well above 0.95 for Accuracy, Balanced accuracy, and f1-macro for all three runs. *Random Forest* performs just slightly below this, while *Logistic Regression* and *Linear SVC* scores approximately in range $0.89 - 0.92$ for these metrics.

Observing class-specific performance scores from the first run, in Table 2 and for all three runs in Appendix B, it can be seen that **none** of the models fails spectacularly. It can be argued that all models actually performs fairly well to really well for all classes, also for class 7 which was a clear minority class. This indicates that the strategy using `f1_macro` as the scoring metric when tuning models and using `class_weight='balanced'` for the classifiers worked as intended.

More so, looking at class-specific performance scores as well as the confusion matrices from the first run in Figure 4, and the second and third run in Appendix A, it can be seen that all classifiers have some problem with class 1, predicting some of those data points as class 2. Looking at the PCA scatterplots in Figure 2, it can be seen that some red dots (class 1) are located within the blue cluster (class 2). This can be a reason why all models seem to predict approximately the same amount of class 1 samples as class 2.

Moving on, all models also have small problems with class 2, predicting some class 2 data points as other classes.

KNN and *SVC with polynomial kernel* have no problems predicting class 5 and class 6, while *Random Forest* has minor problems in doing this. However, *Logistic Regression* and *Linear SVC* are mixing these two classes to a somewhat larger extent. They are also predicting class 6 as class 1 to some extent.

1.4 Conclusions

Particular challenges in this data set and how I addressed them:
Tackling the imbalance of the data set using `f1_macro` as the scoring metric when tuning models and using `class_weight='balanced'` for the classifiers worked as intended, as all models performed well when tasked to predict the minority class.

Overall and class-specific performance of methods:

- All models showed consistent performance over three different runs.
- All models have some problems with class 1, predicting it as class 2.
- All models have minor problems with correctly predicting class 2.

- *KNN* and *SVC with polynomial kernel* are the best performing classifiers. Both have almost perfect performance when predicting classes 3, 4, 5, 6 and 7.
- *Random Forest* classifiers also does a good job, and performs almost as well.

How many features were informative? Did it depended on the method?:

- Data exploration using filtering, scaling, PCA, and performance collection using *Logistic Regression* and *Random Forest* suggested that filtering using ANOVA F-test down to a feature space of range $[15, 30]$ was an appropriate starting point.
- When doing Grid Search using the "large" sample size, $k = 20$ was chosen most frequently by all models except *Random Forest* that most frequently chose $k = 15$.
- It can be stated that all 5 models evaluated found approximately the same number of features as informative.
- Only a small subset of features drives classification. The majority of features are irrelevant or redundant. This suggest that the structure of the problem is lower dimensional.
- ANOVA F-test is univariate and linear, since it works well, the classification boundaries are likely to **not** be strongly nonlinear and likely to **not** be depend on complex feature interactions.

Why might some methods perform better than others on this data set?
Also for different sample sizes?

After selecting features using ANOVA F-test, PCA was applied to transform the data. As PCA prioritizes directions where the data points spread out the most, this changes the geometric space.

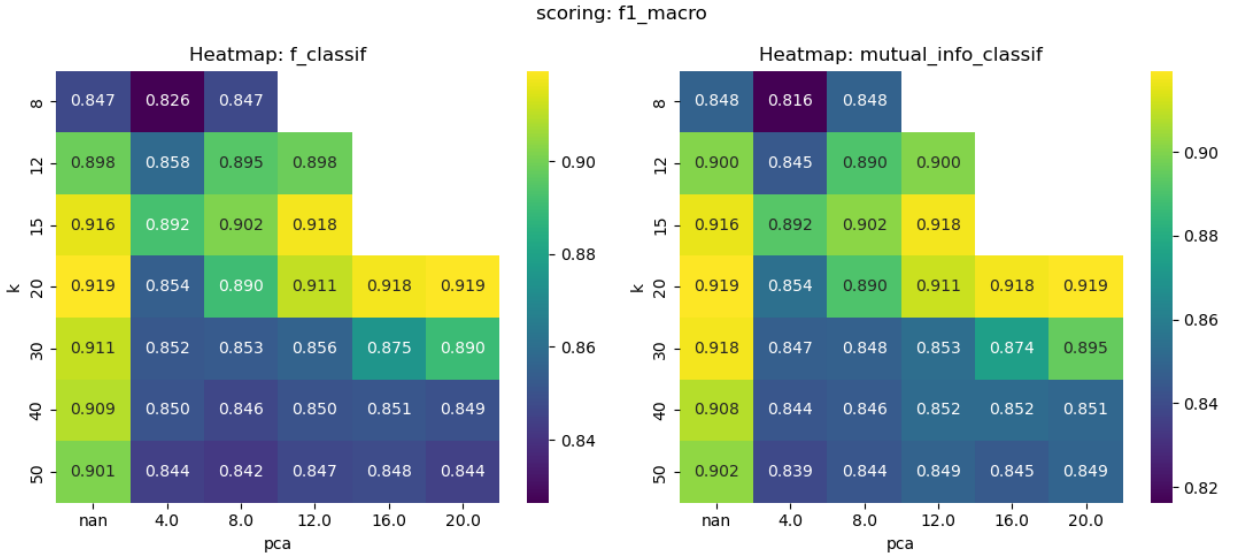
The two best classifiers were *KNN* and *SVC with polynomial kernel*, both are distance-based classifiers. *Linear SVC* is also a distance-based classifier, but it only draws linear decision boundaries. *Random Forest* is an ensemble tree-based model, and *Logistic Regression* is a linear and probabilistic classifier.

The fact that *SVC with polynomial kernel* performed really well, while the performance of *Linear SVC* was possibly the worst among the evaluated classifiers, indicates that the classes in the transformed feature space are not linearly well-separable, but can be separated using smooth nonlinear decision boundaries.

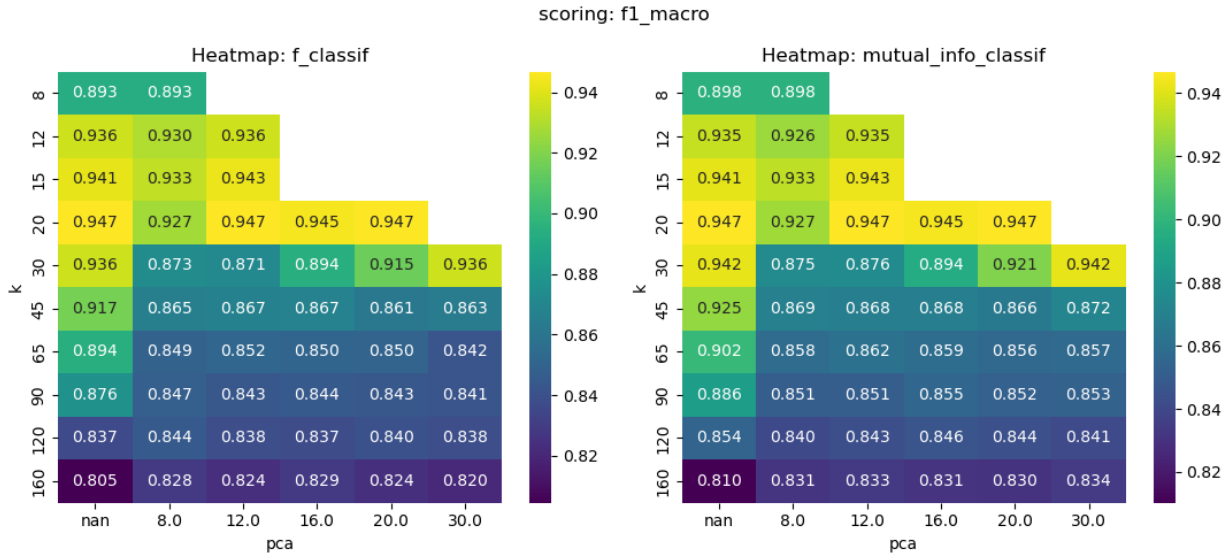
It is likely that in the transformed feature space, distance is meaningful, classes form compact clusters, and class separation requires curved boundaries. Observing the scatterplots of PC's (Figure 2), the above holds true, at least when observing the data projected onto PC1, PC2, PC3 and PC4.

However, looking at the boxplots, it can be seen that *Logistic Regression* and Linear SVC seem to be the most stable, their performance does not increase as clearly when sample sizes gets larger. This indicates that *KNN* and *SVC with polynomial kernel* especially needs more data to learn their neighborhood or curved boundaries, while the linear models *Logistic Regression* and Linear SVC are doing a good job at finding decent linear boundaries even for small sample sizes. However, if the true decision boundaries are non-linear, these models will fail to capture this.

1.5 Figures and Tables



(a) Logistic Regression as scoring classifier. The left heatmap was filter using ANOVA F-test, and the right heatmap was filtered using Mutual Information.



(b) Random Forest as scoring classifier. The left heatmap was filter using ANOVA F-test, and the right heatmap was filtered using Mutual Information.

Figure 1: Data exploration. Evaluating different combinations of filters, filter levels, and PCA. 3000 samples were used, and f1-macro was used as scoring metric.

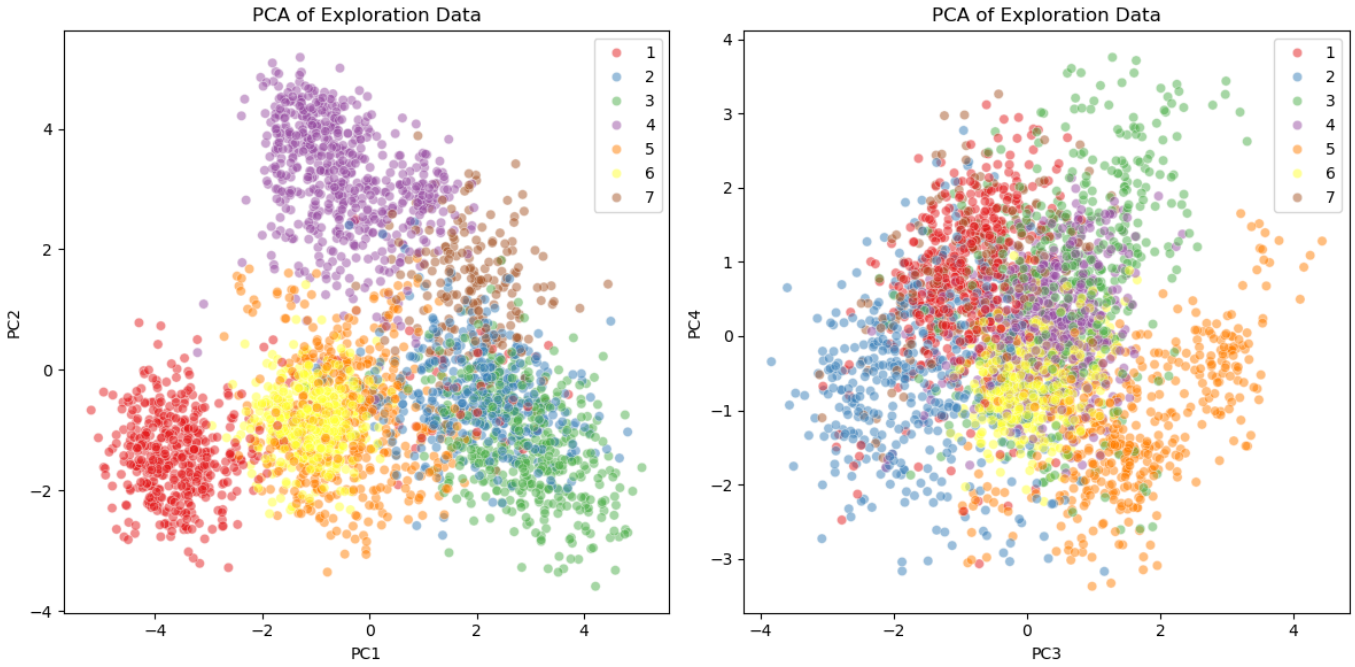


Figure 2: Data exploration. The first 4 principal components, after filtering the exploration data down to 20 features, and applying PCA. 3000 samples were used.

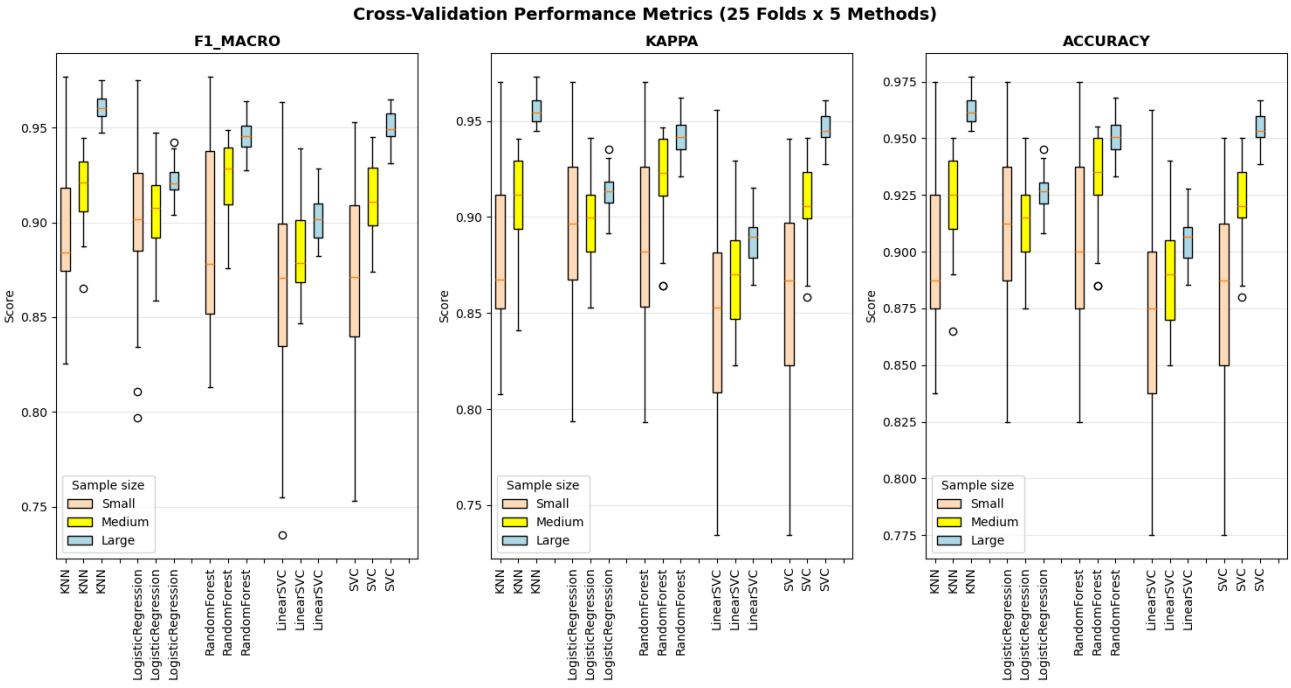


Figure 3: Box-plots made of scores collected during the first of three runs. Box-plots of three performance metrics for five classifiers, trained on different sizes of sample sizes. 25 performance scores are gathered for combination of each model and sample size.

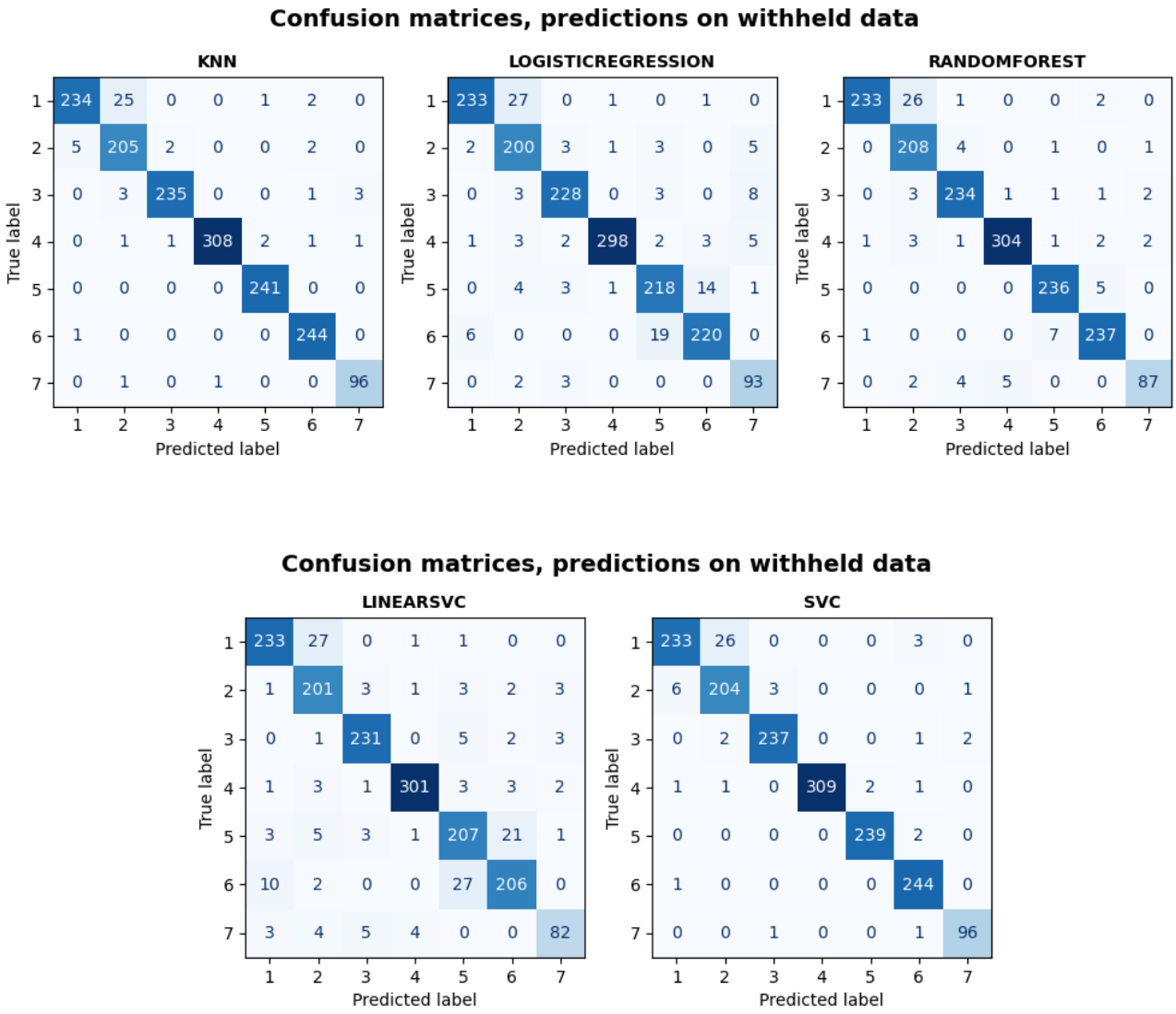


Figure 4: Confusion matrices for the five different classifiers. Based on predictions of the super-test of unseen data in the first of three runs.

Model	Metric	Run 1	Run 2	Run 3
KNN	Accuracy	0.9672	0.9715	0.9740
KNN	Balanced Accuracy	0.9684	0.9711	0.9738
KNN	F1 Macro	0.9661	0.9705	0.9740
Logistic Regression	Accuracy	0.9220	0.9251	0.9332
Logistic Regression	Balanced Accuracy	0.9238	0.9250	0.9310
Logistic Regression	F1 Macro	0.9169	0.9189	0.9274
Random Forest	Accuracy	0.9524	0.9585	0.9573
Random Forest	Balanced Accuracy	0.9472	0.9579	0.9534
Random Forest	F1 Macro	0.9479	0.9565	0.9551
Linear SVC	Accuracy	0.9041	0.9121	0.9189
Linear SVC	Balanced Accuracy	0.8969	0.9061	0.9102
Linear SVC	F1 Macro	0.8981	0.9076	0.9131
SVC	Accuracy	0.9666	0.9678	0.9734
SVC	Balanced Accuracy	0.9676	0.9667	0.9719
SVC	F1 Macro	0.9658	0.9653	0.9719

Table 1: Model performance on unseen data (super-test), across three runs.

Model	Class	Precision	Recall	F1-score	Support
KNN	1	0.97	0.89	0.93	262
	2	0.87	0.96	0.91	214
	3	0.99	0.97	0.98	242
	4	1.00	0.98	0.99	314
	5	0.99	1.00	0.99	241
	6	0.98	1.00	0.99	245
	7	0.96	0.98	0.97	98
Logistic Regression	1	0.96	0.89	0.92	262
	2	0.84	0.93	0.88	214
	3	0.95	0.94	0.95	242
	4	0.99	0.95	0.97	314
	5	0.89	0.90	0.90	241
	6	0.92	0.90	0.91	245
	7	0.83	0.95	0.89	98
Random Forest	1	0.99	0.89	0.94	262
	2	0.86	0.97	0.91	214
	3	0.96	0.97	0.96	242
	4	0.98	0.97	0.97	314
	5	0.96	0.98	0.97	241
	6	0.96	0.97	0.96	245
	7	0.95	0.89	0.92	98
Linear SVC	1	0.93	0.89	0.91	262
	2	0.83	0.94	0.88	214
	3	0.95	0.95	0.95	242
	4	0.98	0.96	0.97	314
	5	0.84	0.86	0.85	241
	6	0.88	0.84	0.86	245
	7	0.90	0.84	0.87	98
SVC	1	0.97	0.89	0.93	262
	2	0.88	0.95	0.91	214
	3	0.98	0.98	0.98	242
	4	1.00	0.98	0.99	314
	5	0.99	0.99	0.99	241
	6	0.97	1.00	0.98	245
	7	0.97	0.98	0.97	98

Table 2: Comparison of class specific performance on unseen data. Based on the predictions of supertest from the first of three runs.

2 Disruption of the given data

Make some models perform worse using a minimum disruption of the given data set.

2.1 Ideas and reasoning

From above, I have found that that only a small subset of features provide sufficient information and can be used for great classification performance. Approximately 15-20-30 features are really useful, and can be obtained by ANOVA F-test. PCA is also applied, which creates a new feature space in directions of maximum variance. Since *KNN* was one of two best performing classifiers, we know that the local geometry matters.

As I look for a targeted, minimal disruption that exploits what the models and the data depend on, I thought it would be a good idea to somehow try to alter some of the feature vectors that are of highest importance.

My first thought was to re-scale or somehow distort some important feature vectors. However, since standard scaling and PCA is applied in the pipeline, it is of no use to try to distort by scaling or by adding constants to features, as standard scaling and PCA will absorb such distortions.

My next idea was to shuffle data within the most informative features of the original data. The approach meant that from the training data, a number of the most informative features were extracted using ANOVA F-test. For each informative feature vector, the data values are shuffled within the feature vector itself. This means that the data within each feature vector are the same, but locations have switched.

This disruption keeps the distribution of values intact within each feature vector, and it will keep the scale of data intact in each feature vector. It will however, destroy the relationship between labels and the targeted feature vectors. This type of distortion will not be canceled out by scaling or PCA.

This approach should break the local geometry structure for the most informative feature vectors. The local geometry structure is especially important for *KNN*, and therefore the performance of *KNN* is expected to decrease more than the performance of other classifiers.

More so, since only a few of the total 820 feature vectors will be altered, I expect that some global structure remain in the data, and therefore, linear and more rigid models such as *Linear SVC* and *Logistic Regression* should not fail as drastically as *KNN*.

Finally, *Random Forest* which is an ensemble method and should be relatively robust, since it is not really concerned with neighborhoods nor distance.

2.2 Process

The below process was repeated three times. I did a train-test-split, saving 20% of the data for testing and collecting performance scores. ANOVA F-test was applied to the training data, and the indices for the 20 most informative features were extracted. From these 20 features, I randomly chose 15 of them. For each of these feature vectors, the data was shuffled within each feature vector. This was also done to the test data separately. This means that only 15 out of 820 feature vectors were altered. No noise was added, and no labels were changed. That is really a minimal disruption if you ask me.

After this, all five models described in the Section 1 were fitted to the disrupted training data, and tasked to predict the disrupted test data. Confusion matrices were plotted and scores were collected.

2.3 Performance evaluation

Overall performance scores for all models can be seen in Table 3. Looking at the overall performance, it can be seen that *Random Forest* performed best, and *KNN* performed worst, just as expected.

Random Forest performs better than the other models for almost all metrics and all runs, looking at the overall performance scores, while *KNN* performed worse than all other models for all metrics and all runs, looking at overall performance scores.

The linear models, *Logistic Regression* and *Linear SVC* both outperformed *SVC with polynomial kernel*. Therefore, we have reached a state where the two models that performed best on the original data are the ones performing worst on the disrupted data.

Confusion matrices from the first of three runs can be seen in Figure 5. Confusion matrices from the second, and the third run can be seen in Appendix A

As *KNN* and *Random Forest* performed furthest apart, class-specific performance score for these models can be seen in Table 4, and in Table 5 respectively.

Looking at class-specific scores, there is a lot more variability between runs in this scenario, compared to when the models were applied to original data. More so, *Random Forest* seem to struggle the most when it comes to correctly predicting class 5 and 7, as the Recall-scores for these classes are in range $0.38 - 0.54$ and $026. - 0.51$ respectively. However, these classes also seem to be the hardest to predict for *KNN*, which performs way worse.

2.4 Conclusions

The disruption chosen is specific to my data, since only a few features (15-30), are important for classification. Therefore, disrupting a subset of them will significantly impact performance. This approach would not necessarily work for a datasets where the true signal is distributed across more features.

KNN depends on the local geometry and local similarity. Shuffling values within important features will disrupt neighborhood structure, causing *KNN* to have incorrect nearest neighbors. Leading to large drops in performance.

Random Forest is more resilient to this disruption because it does not rely on distance metrics or local geometry. Instead, it constructs multiple decision trees using subsets of features. Even when some of the most informative features are shuffled and lose their predictive power, the model can still rely on the remaining informative features.

The linear models *Logistic Regression* and *Linear SVC* rely directly on individual features. They seem to be able to learn weights that puts greater emphasis on the informative features that remains, and therefore, these methods do not experience a severe drop in performance.

2.5 Figures and Tables

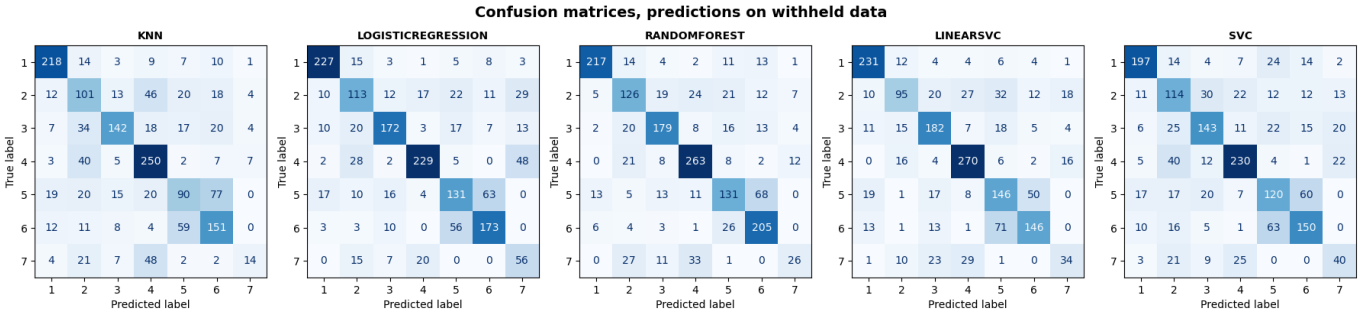


Figure 5: Confusion matrices from the first of three runs on disrupted data.

Model / Metric	Run1	Run2	Run3
KNN			
accuracy	0.597772	0.529084	0.506188
balanced_accuracy	0.545658	0.491703	0.461505
f1_macro	0.544784	0.489452	0.455857
Logistic Regression			
accuracy	0.681312	0.671411	0.618193
balanced_accuracy	0.665087	0.669847	0.615748
f1_macro	0.657301	0.657144	0.600926
Random Forest			
accuracy	0.709777	0.686262	0.676361
balanced_accuracy	0.662841	0.665231	0.640152
f1_macro	0.665036	0.669246	0.639969
Linear SVC			
accuracy	0.683168	0.652228	0.603342
balanced_accuracy	0.640887	0.630470	0.566850
f1_macro	0.641856	0.620084	0.553043
SVC			
accuracy	0.615099	0.601485	0.564975
balanced_accuracy	0.589478	0.584437	0.540646
f1_macro	0.590559	0.584164	0.538083

Table 3: Performance metrics across three runs for multiple classifiers, on disrupted data

KNN Metric / Class	Run1	Run2	Run3
Precision			
1	0.792727	0.634812	0.451895
2	0.419087	0.540909	0.555556
3	0.735751	0.520179	0.547264
4	0.632911	0.565321	0.772989
5	0.456853	0.397260	0.229167
6	0.529825	0.431227	0.358268
7	0.466667	0.500000	0.400000
Recall			
1	0.832061	0.709924	0.591603
2	0.471963	0.556075	0.630841
3	0.586777	0.479339	0.454545
4	0.796178	0.757962	0.856688
5	0.373444	0.240664	0.182573
6	0.616327	0.473469	0.371429
7	0.142857	0.224490	0.142857
F1-score			
1	0.811918	0.670270	0.512397
2	0.443956	0.548387	0.590810
3	0.652874	0.498925	0.496614
4	0.705219	0.647619	0.812689
5	0.410959	0.299742	0.203233
6	0.569811	0.451362	0.364729
7	0.218750	0.309859	0.210526

Table 4: KNN class-wise precision, recall, and F1-score across three runs, on disrupted data

RandomForest Metric / Class	Run1	Run2	Run3
Precision			
1	0.893004	0.860656	0.665517
2	0.580645	0.694064	0.665306
3	0.755274	0.635628	0.682403
4	0.769006	0.722063	0.847953
5	0.612150	0.631868	0.522727
6	0.654952	0.582492	0.594796
7	0.520000	0.641026	0.590164
Recall			
1	0.828244	0.801527	0.736641
2	0.588785	0.710280	0.761682
3	0.739669	0.648760	0.657025
4	0.837580	0.802548	0.923567
5	0.543568	0.477178	0.381743
6	0.836735	0.706122	0.653061
7	0.265306	0.510204	0.367347
F1-score			
1	0.859406	0.830040	0.699275
2	0.584687	0.702079	0.710240
3	0.747390	0.642127	0.669474
4	0.801829	0.760181	0.884146
5	0.575824	0.543735	0.441247
6	0.734767	0.638376	0.622568
7	0.351351	0.568182	0.452830

Table 5: Random Forest class-wise precision, recall, and F1-score across three runs, on disrupted data

3 Mislabeled data

3.1 Any evidence of mislabeling in the given dataset?

Samples that potentially are mislabeled are samples that are outliers in their own class, or samples that sit inside regions dominated by another class.

I have some suspicions of the existence of mislabeled data based on:

- In Figure 2, scatterplot of PC1, PC2, PC3 and PC4 of exploration data. It can clearly be seen that some red dots (class 1) are distributed in and around the cluster of blue dots (class 2).
- Confusion matrices stemming from Section 1.5 shows that all models for all runs, seem to have equal problems in correctly classifying the same number of class 1 data point, and instead predict them as class 2.
- My suspicion is that some class 2 data points are mislabeled as class 1.

My previous results have shown strong indications of feature selection being able to pick up on strong and true signal, and more interestingly, that *KNN* works really well, which indicates that the filtered and PCA-transformed data experience fairly separable and local neighborhoods.

The key idea for detecting potentially mislabeled data is to use *KNN* to find the k-nearest-neighbors of each data point, and then flag data points as suspicious if their nearest neighbors mostly belong to different classes than themselves.

This approach should work for my data because it has been shown that the transformed data set has strong local structure, and classes seem to be clustered. Therefore, a clearly mislabeled point should not agree with its local environment.

3.2 Process

I used all data from `X_TR` and `y_TR`. I filtered the data using ANOVA F-test and extracted the 20 most informative features, scaled the data, and applied PCA-transformation, exactly as previously have been done.

I fitted a *KNN* classifiers to this data, using `n_neighbors=5`, since this hyperparameter was the most common choice from when I was tuning my models in Section 1.

Then, for each data point, the indices of its 5 nearest neighbors were saved. Then, for each data point, it was checked which classes its 5 nearest neighbors had, and an agreement score was calculated. In this scenario, using `n_neighbors=5`, the agreement scores can be $\{0,\ 0.2,\ 0.4,\ 0.6,\ 0.8,\ 1\}$, corresponding to the fraction of neighbors sharing the same class label as the point itself. If a data point received an agreement score less than 0.39 it was flagged as a suspicious, meaning that this point can be considered mislabeled.

The results can be seen in Table 6. A total of 92 points were flagged as suspicious. Out of them, 72 were labeled as class 1, and 13 were labeled as class 2. This indicates that my hypotheses that some points that actually are class 2 have been labeled as class 1 might well be true.

The fact that 13 points from class 2 are flagged as suspicious could be due to mislabeling, but it could also be a case of these points being outliers, and therefore have neighbors not being of their own class.

3.3 Verification of approach

I artificially mislabeled data by randomly switching labels for 5% of the samples. Each switched labeled was uniformly picked from $\{1, 2, 3, 4, 5, 6, 7\}$, which means that there was some chance for each switched label to become its original label. If a label was switched but happened to get its original value back, it will not be possible to detect that.

Given there are different proportions to the classes, and switched labels are picked uniformly, it is perhaps reasonable to assume that $\frac{1}{8}$ of the switched labels became its original label. I mislabeled 404

samples. Therefore, a good validation would detect around $404 \cdot \frac{7}{8} \approx 354$ mislabeled samples.

The detection was run in the same way as described above, and the results came back as:

- Number of labels switched: 404
- Number of these switched labels detected: 332

Therefore, it can be verified that this method of detecting mislabeled samples works as intended. This also means that it is very likely that there are mislabeled data in the given dataset. Especially amoing data points labeled as class 1, but in reality most likely belonging to class 2.

3.4 Conclusions

Using a KNN-based neighbor agreement score, I identified samples whose labels disagree with the majority of their nearest neighbors. These points are likely mislabeled or outliers, as the dataset have been shown to experience a strong local structure in general.

This approach should work well for my data set, since previous results have shown that *KNN* achieves strong performance, indicating that samples of the same class tend to be located close to each other in feature space.

- It can be concluded that it is very likely that my data set contains some mislabeled samples.
- Samples labeled as class 1 constituted of 78% of all samples flagged as possibly mislabeled.
- Samples labeled as class 2 constituted of 14% of all samples flagged as possibly mislabeled. These samples are either mislabeled or are true outliers.
- The method used to detect mislabel samples was successfully verified.

3.5 Figures and Tables

Class label	# Flagged as suspicious
1	72
2	13
3	3
4	1
6	2
7	1

Table 6: Class labels and counts of detected data points that are suspected to be mislabeled.

4 ML pipeline for a super-majority class

4.1 Data pre-procssing

I will use the given data and create a much more challenging problem where one class will be a super-majority class, and the other classes will be tiny minority classes.

As class 4 had the most samples, I kept all 1568 samples, using this as the majority class. All other classes were randomly down-sampled such that only 16 samples were remaining for each of these classes.

This resulted in a dataset of 1664 samples. Approximately 94.2% of data belonged to class 4. Each minority class constituted of less than 1% of all samples.

I ran this data through my pipeline, including filtering using ANOVA F-test, standard scaling, and PCA. All is well described under in Section 1. I still use `f1_macro` as the scoring metric and `class_weight='balanced'` for all classifiers where this option was available.

4.2 Process

The below process was run one time. Starting with randomly extracting 16 samples for each minority class. These samples were used together with all indices corresponding to class 4, to form a new dataset with 1664 samples.

Thereafter, all of this data was used to iteratively train and evaluate models and collect performance metrics used to make boxplots. I only used one sample size, 1600. This gives 1280 samples for training and 320 for testing for each fold. Collecting performance metrics was done iteratively, as well as gathering information about the best sets of hyperparameters. This process provided 25 performance evaluations for each model.

After this procedure, the set of hyperparameters that was chosen most often by Grid Search for each model were extracted, and fitted to new pipelines. These pipelines were fitted to all training data, and thereafter used to predict the labels from the test set. The test data constituted of 1616 randomly sampled data points from `X_TR`. Meaning that there was **no** severe imbalance in the test data.

4.3 Performance evaluation

Performance can be seen as Boxplots in Figure 6. From the boxplots it can be seen that all models perform really well on accuracy. This does not tell much about the actual performance, as high accuracy when being evaluated on imbalanced data can be achieved by always predicting the majority class. However, looking at the f1-macro scores, and Kappa scores, it can be seen that all models perform fairly well. Surprisingly well, I would argue. It can also be seen that *KNN* and *SVC with polynomial kernel* performs the best. Just like they did for in Section 1. However, the boxplots are based on evaluations of severely imbalanced data.

Evaluations of unseen, and none-imbalanced data can be seen in the confusion matrices in Figure 7, and in Table 7, where overall performance metrics are gathered. All scores are fairly good for all the models, but *SVC with polynomial kernel* and *Logistic Regression* performs the best for this set of test data.

4.4 Using SMOTE

The same procedure as above was also done one time using Synthetic Minority Oversampling (SMOTE). Meaning that all class 4 samples were kept, while all other classes were decreased to only 16 samples each. After this, SMOTE was applied, oversampling all minority classes such that the resulting training set had 1568 samples of each class. This data was then used as training data, for hyperparameter tuning and so on.

Predictions were made on the same set of 1616 randomly sampled data points as was also used as the test set in Section 4.1. Evaluations of this unseen, and none-imbalanced data can be seen in the confusion matrices in Figure 8, and in Table 8.

It can be seen that oversampling using SMOTE increased performance of *KNN* a lot. This could be down to the fact that when having oversampled training data, there are more neighbors with the

same label for the minority classes, meaning that *KNN* can more effectively construct neighborhoods. Which I suppose must have been really hard when minority classes only had 16 samples each in the imbalanced data.

Other models does however perform worse after being trained on oversampled data. *Linear SVC* and *Logistic Regression* performs worse. It may be possible that the synthetic oversampled data distorts the linear boundaries these models had found from true signal, and therefore performance drops a bit.

4.5 Conclusions

Since I only executed these two approaches for one run each, I should not draw any drastic conclusions. However, I think it can be said that the results indicates that my original pipeline was well constructed, as it seem to be able to handle severe class imbalance and still perform fairly well.

Oversampling using SMOTE increased the performance of *KNN* a lot. The performance scores for *KNN* are way better than the other classifiers, see Table 8. This is probably because *KNN* had a hard time to find neighbors when all minority classes in the training set were only 16 samples each. Oversampling these minority classes made it possible for *KNN* to find neighborhoods and make better decision boundaries.

In Section 1 it was seen that some models had some problems mixing up class 5 and class 6, while *KNN* and *SVC with polynomial kernel* did not have this problem.

Here, not surprisingly, it can be seen that all models have even more problems with these two classes. However, what is interesting is that the different models are predicting class 5 and class 6 with different success, indicating that the decision boundary between these two classes are captured differently across the models, some more successful than others.

4.6 Figures and Tables

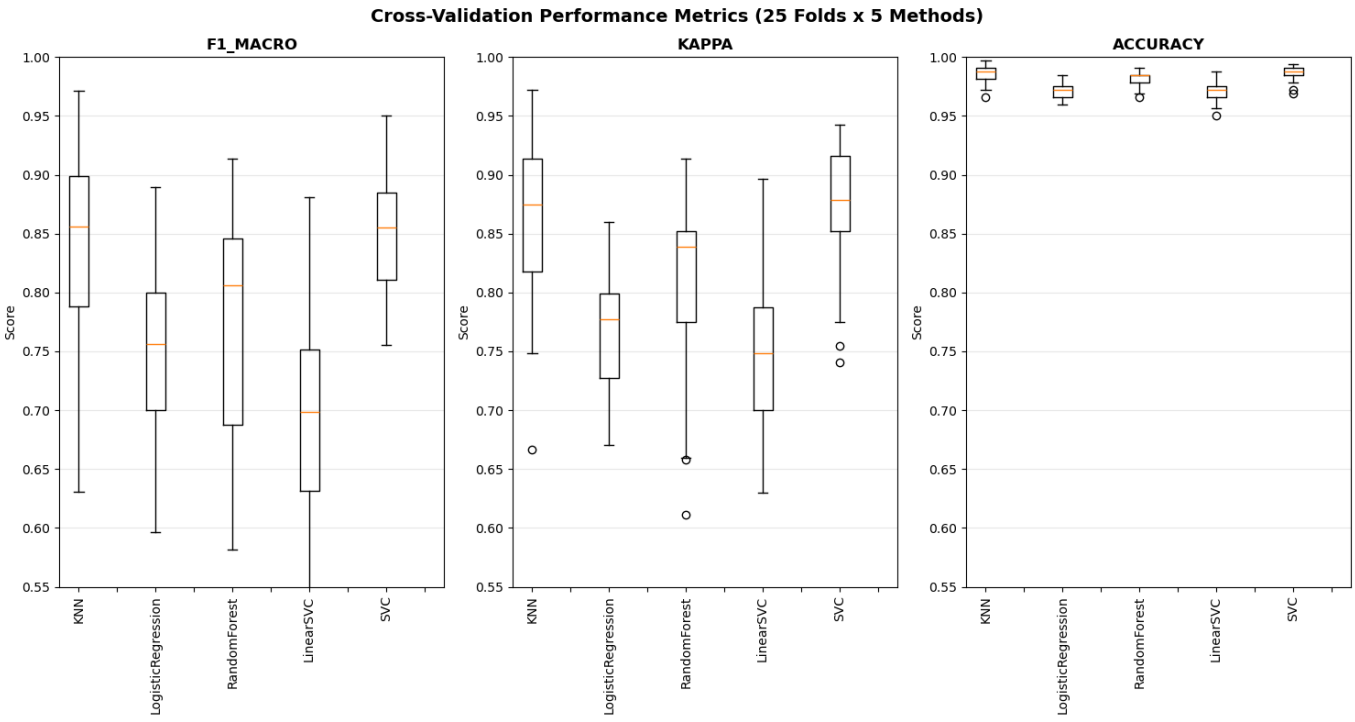


Figure 6: Boxplots of performance scores, collected iteratively, models were trained and evaluated on severely imbalanced data.

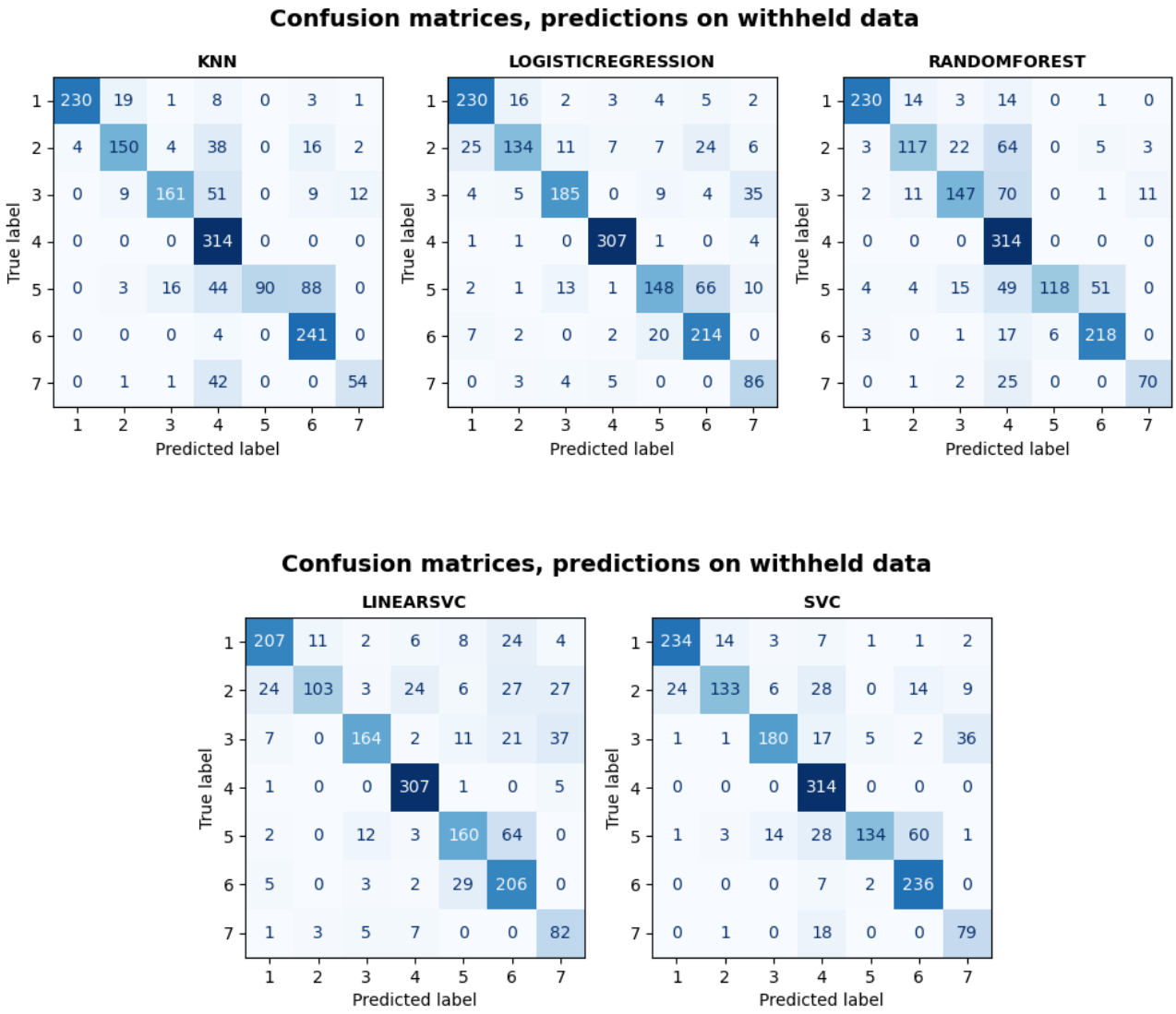


Figure 7: Confusion matrices for the five different classifiers trained on severely imbalanced data. Confusion matrices are based on predictions of randomly sampled and non-imbalanced unseen data.

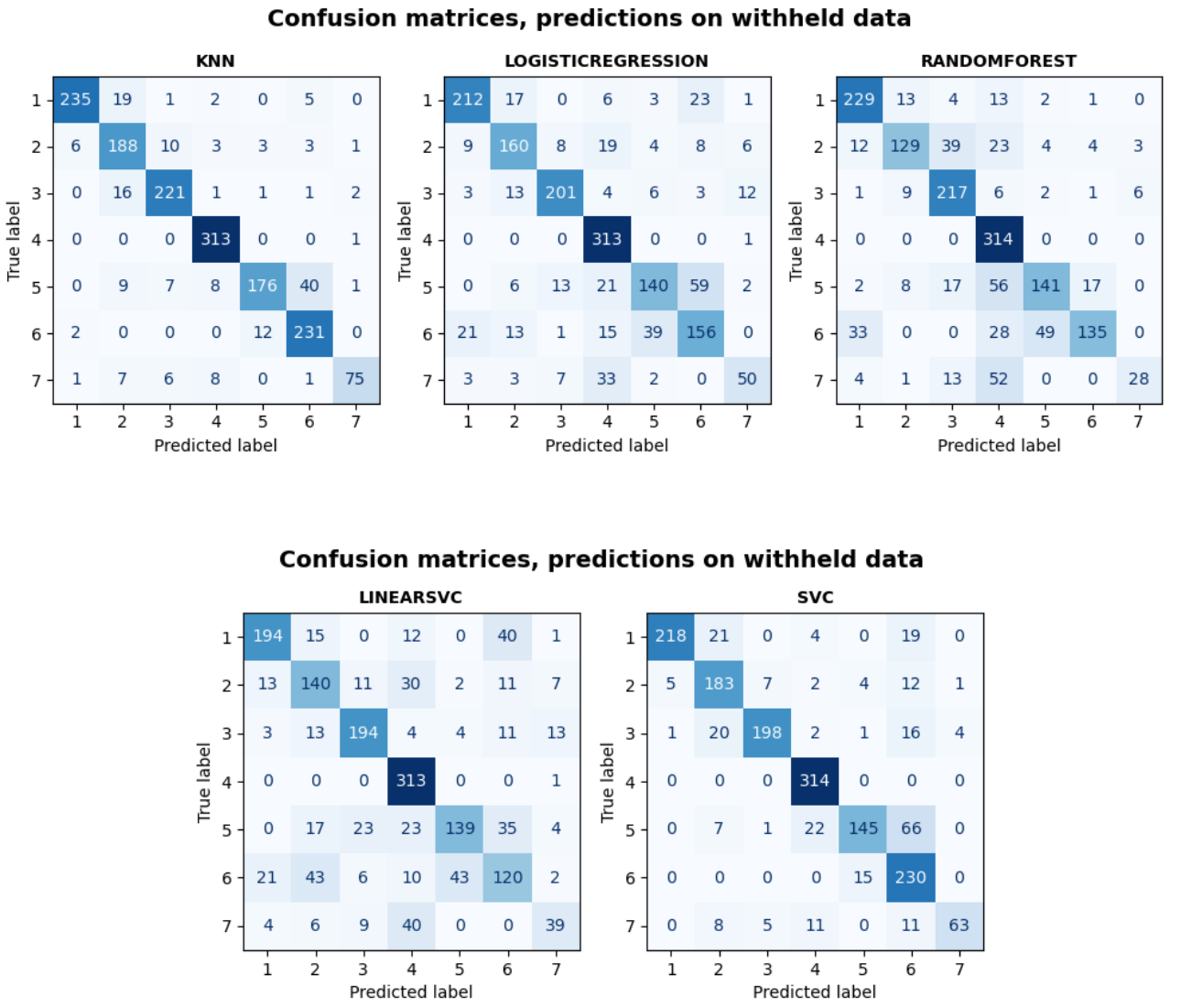


Figure 8: Confusion matrices for the five different classifiers trained on SMOTE oversampled data. Confusion matrices are based on predictions of randomly sampled and non-imbalanced unseen data.

Model	Accuracy	Balanced Accuracy	F1-macro
KNN	0.7673	0.7360	0.7435
LogisticRegression	0.8069	0.8016	0.7884
RandomForest	0.7512	0.7322	0.7455
LinearSVC	0.7605	0.7526	0.7388
SVC	0.8106	0.7977	0.7910

Table 7: Overall performance scores for all classifiers. Trained on severely imbalanced data. Predictions made on non-imbalanced unseen data.

Model	**Accuracy**	**Balanced Accuracy**	**F1-macro**
KNN	0.8905	0.8748	0.8807
LogisticRegression	0.7624	0.7303	0.7373
RandomForest	0.7382	0.6850	0.6936
LinearSVC	0.7048	0.6654	0.6693
SVC	0.8360	0.8127	0.8227

Table 8: Overall performance scores after oversampling. Trained on data that first was severely imbalanced, but then oversampled using SMOTE. Predictions made on non-imbalanced unseen data.

5 Tackle class imbalance using hierarchical classification

One way to approach a problem where one super-majority class exists and the other classes are tiny minorities is to first run a majority-vs-rest classifier, and then a multiclass problem for the minority classes. The approach will be devloped and implemented in this section.

5.1 Data selection and pre-processing

I used the same data approach as in Section 4, i.e. having class 4 as the majority class, and randomly generating 16 samples from each minority class. These were used together with all samples of class 4. This resulted in 1664 samples.

Then, I created binary labels, making majority class 4 get the binary label 0, and all minority classes got binary label 1. I created a data pre-procesing pipeline with the most commonly selected parameters from Section 1, namely ANOVA F-tilter down to 20 features, standard scaling, and PCA.

5.2 Stage 1

The imbalanced training data was pre-processed using the pre-processing pipeline. A 10-fold `GridSearchCV` was applied to the training data. Evaluating the five different models with different combinations of hyperparameters. Scoring metric used was `f1_macro` as previously. The best performing model for the *majority-vs-rest* approach was *SVC with polynomial kernel*, degree=2.

5.3 Stage 2

Minority labels and minority training data samples were now put back together. A 10-fold `GridSearchCV` was applied to this remaining data. Scoring metric was `f1_macro` as previously. The best performing model for the *multiclass classification for minority classes* approach was *Logistic Regression*.

5.4 Stage 3

Creation of a hierarchical prediction function that can take raw X data, apply the pre-processing pipeline, predict *majority-vs-rest* using the Stage 1 classifier *SVC with polynomial kernel*, and then predict *multiclass classification for minority classes* using *Logistic Regression*.

5.5 Performance evaluation

Predictions were made on the same set of 1616 randomly sampled data points that were used in Section 4, making it possible to compare performance across.

Evaluations of this unseen, and none-imbalanced data can be seen in the confusion matrices in Figure 9, and in Table 9. Class-specific performance scores can be seen in Table 10.

It can be said that the overall performance looks good, with overall accuracy, balanced accuracy and F1-macro all being in range $[0.78 - 0.82]$. Which is a better performance than almost all the models evaluated on the same test data in Section 4. Therefore, it can be said that the hierarchical classification model worked really well for my data.

Looking at class-specific scores, my hierarchical model seem to have most problems with correctly predicting class 7. The class-specific Recall and F1-scores for class 7 is significantly lower than for all other classes. I cannot think of a reason why this class performs worse than the others. However, looking at confusion matrices from Section 4.6, it can be seen some of the those models performed even worse or at the same magnitude, when it comes to predicting class 7 for this test set.

5.6 Conclusions

- Classification models were chosen for the "majority-vs-rest" step and the "multiclass classification for minority classes" step by using `GridSearchCV`.
- The above approach resulted in *SVC with polynomial kernel* being the best classifier for "majority-vs-rest"

- While *Logistic Regression* being the best classifier for "multiclass classification for minority classes"
- Hierarchical classification model worked really well for my data
- Overall performance for my Hierarchical classification model was better than 8 out of 10 of the models that were evaluated in Section 4. Where only *KNN* and *SVC with polynomial kernel* trained on over-sampled SMOTE data can be seen to perform better on overall performance scores.

5.7 Figures and Tables

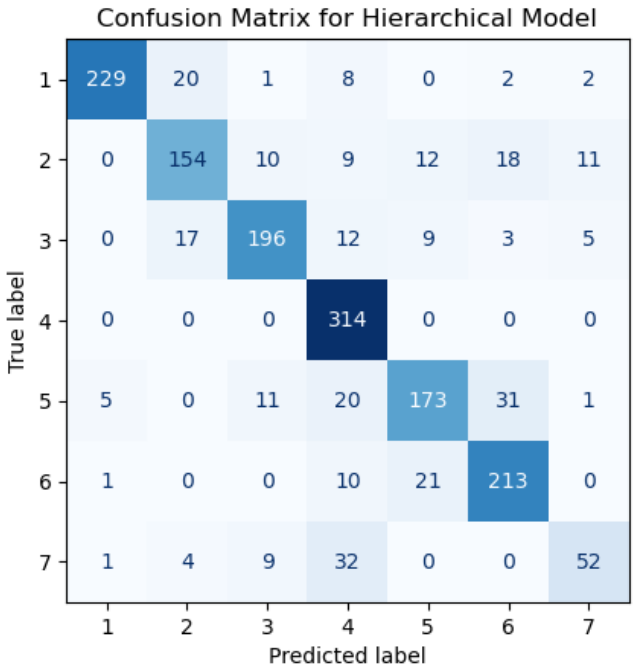


Figure 9: Confusion matrix based on predictions by hierarchical prediction model on randomly sampled and non-imbalanced unseen data.

Model	Accuracy	Balanced Accuracy	F1-macro	Recall-macro
Hierarchical Model	0.8236	0.7888	0.7983	0.7888

Table 9: Performance metrics for the hierarchical classification model on randomly sampled and non-imbalanced unseen data.

Class	Precision	Recall	F1-score	Support
1	0.97	0.87	0.92	262
2	0.79	0.72	0.75	214
3	0.86	0.81	0.84	242
4	0.78	1.00	0.87	314
5	0.80	0.72	0.76	241
6	0.80	0.87	0.83	245
7	0.73	0.53	0.62	98

Table 10: Per-class precision, recall, F1-score, and support from predictions made by hierarchical prediction model on randomly sampled and non-imbalanced unseen data.

A Figures - for the curios reader

A.1 Question 1 - Construct a ML pipeline

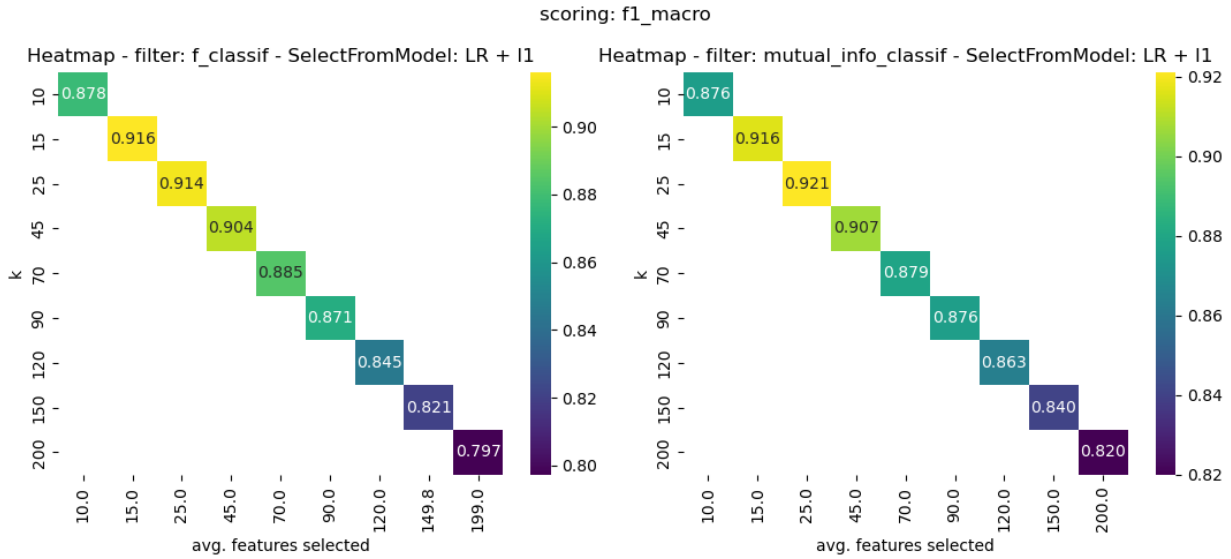


Figure 10: Logistic Regression as scoring classifier. The left heatmap was filter using ANOVA F-test, and the right heatmap was filtered using Mutual Information. Average feature selected refers to the average features selected after applying Logistic Regression with L1-penalty. As can be seen, this approach did not decrease the feature space for this data set. Scores are average f1-macro scores.

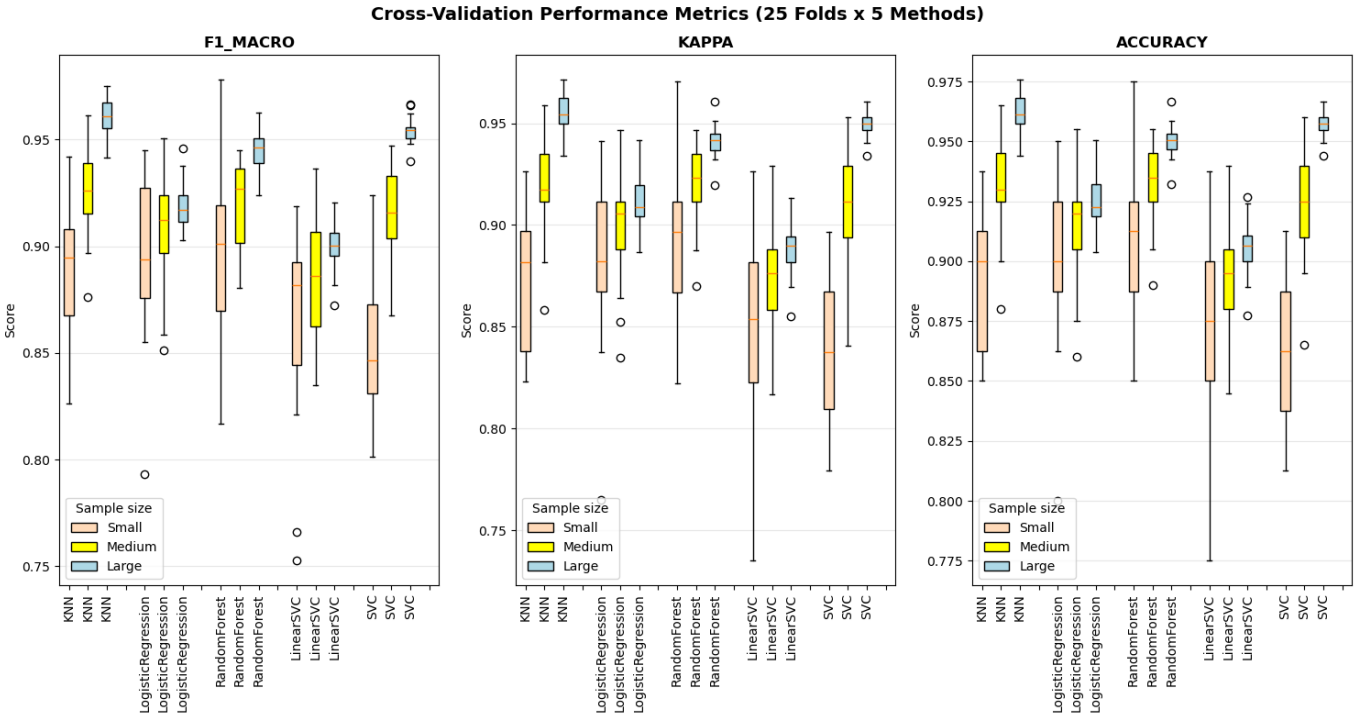


Figure 11: Box-plots made of scores collected during the **second** of three runs. Box-plots of three performance metrics for five classifiers, trained on different sizes of sample sizes. 25 performance scores are gathered for each model + sample size.

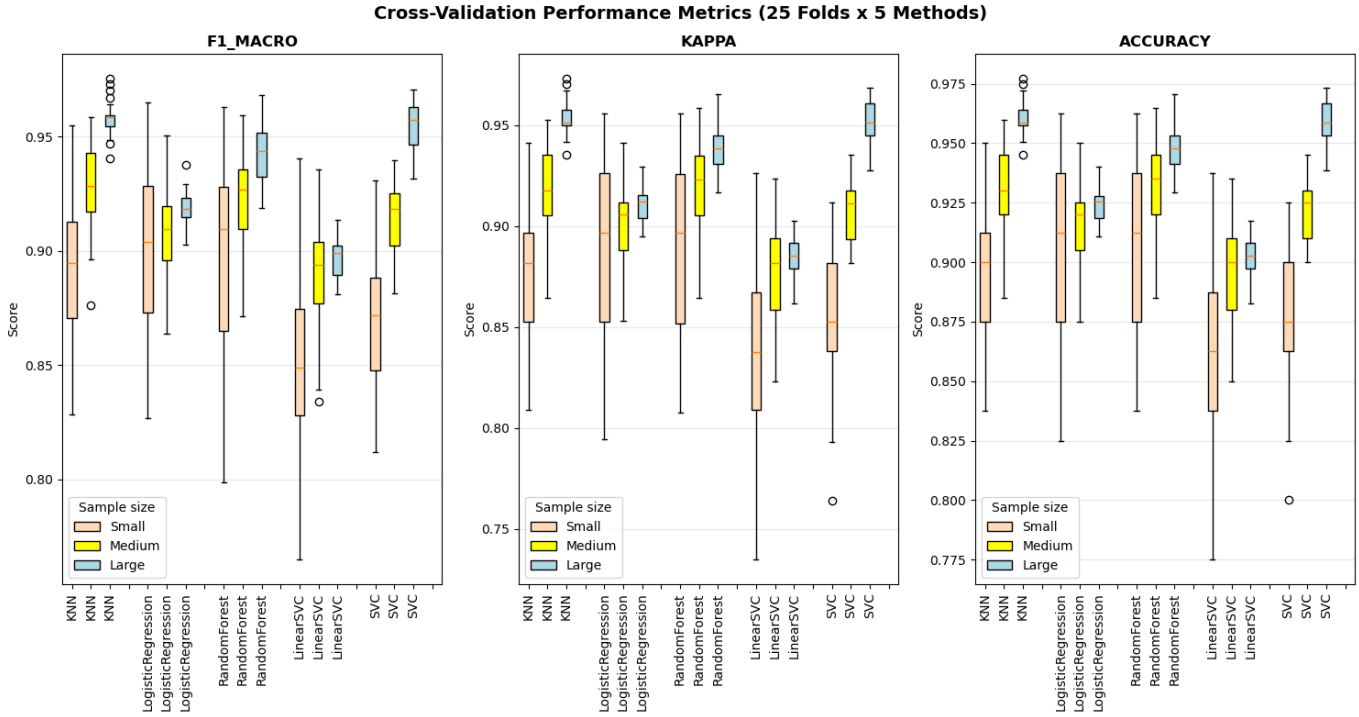


Figure 12: Box-plots made of scores collected during the **third** of three runs. Box-plots of three performance metrics for five classifiers, trained on different sizes of sample sizes. 25 performance scores are gathered for each model + sample size.

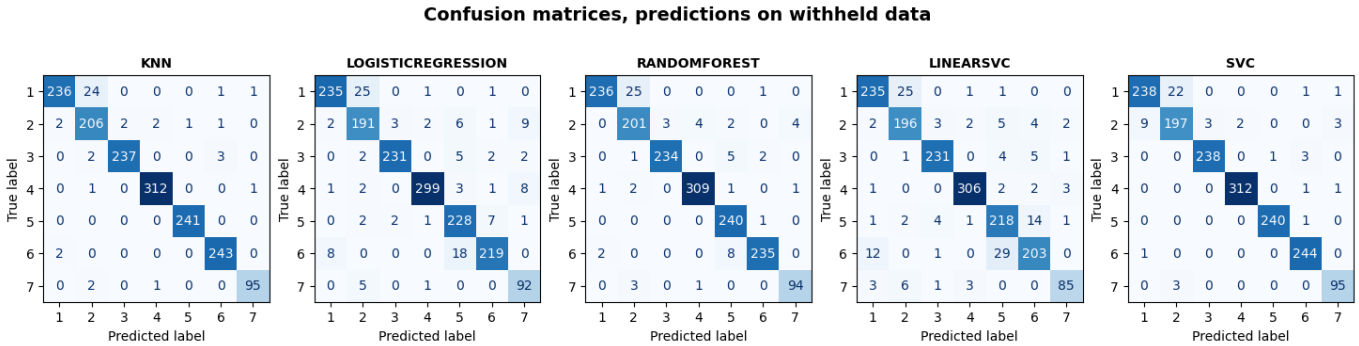


Figure 13: Confusion matrices for the five different classifiers. Based on predictions of the super-test of unseen data in the **second** of three runs

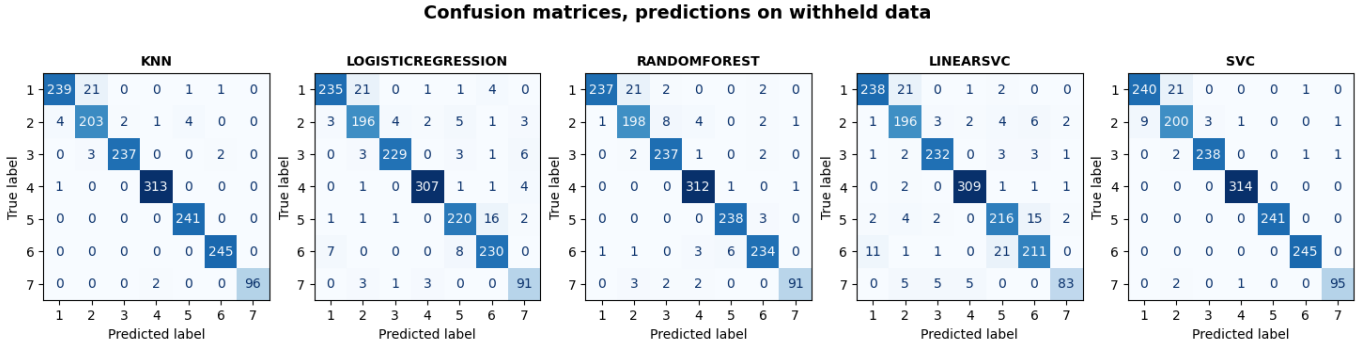


Figure 14: Confusion matrices for the five different classifiers. Based on predictions of the super-test of unseen data in the **third** of three runs

A.2 Question 2 - Disruption of the given data

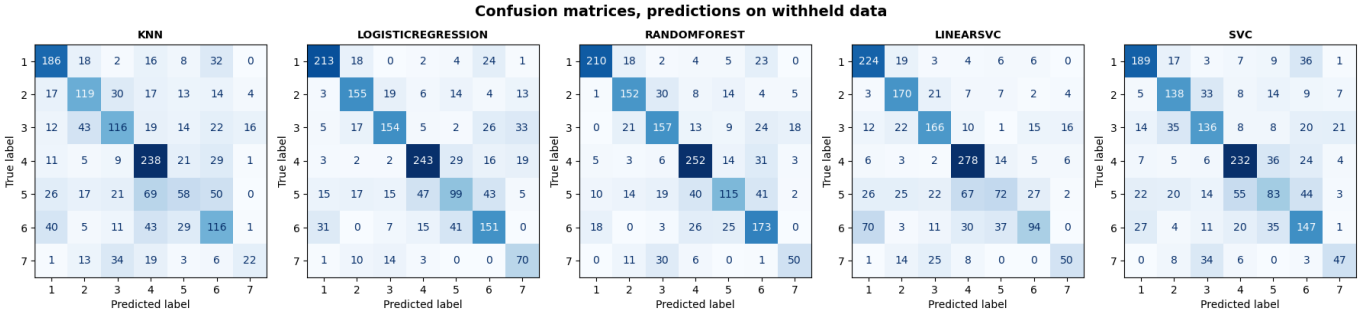


Figure 15: Confusion matrices from the **second** of three run on **disrupted data**.

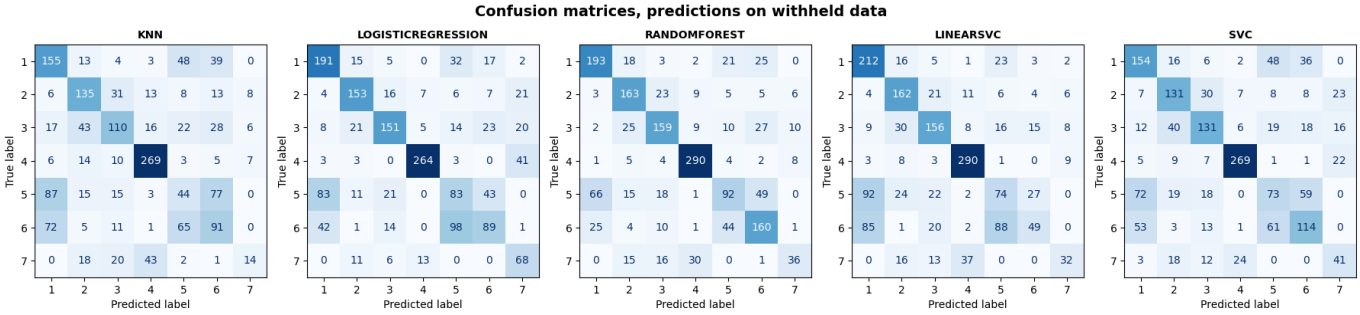


Figure 16: Confusion matrices from the **third** of three run on **disrupted data**.

B Tables - for the even more curios reader

B.1 Question 1 - Construct a ML pipeline

Class	Precision			Recall			F1-score		
	Run1	Run2	Run3	Run1	Run2	Run3	Run1	Run2	Run3
1	0.9750	0.9833	0.9795	0.8931	0.9008	0.9122	0.9323	0.9402	0.9447
2	0.8723	0.8766	0.8943	0.9579	0.9626	0.9486	0.9131	0.9176	0.9206
3	0.9874	0.9916	0.9916	0.9711	0.9793	0.9793	0.9792	0.9854	0.9854
4	0.9968	0.9905	0.9905	0.9809	0.9936	0.9968	0.9888	0.9921	0.9937
5	0.9877	0.9959	0.9797	1.0000	1.0000	1.0000	0.9938	0.9979	0.9897
6	0.9760	0.9798	0.9879	0.9959	0.9918	1.0000	0.9859	0.9858	0.9939
7	0.9600	0.9794	1.0000	0.9796	0.9694	0.9796	0.9697	0.9744	0.9897

Table 11: KNN class-wise performance across runs

Class	Precision			Recall			F1-score		
	Run1	Run2	Run3	Run1	Run2	Run3	Run1	Run2	Run3
1	0.9628	0.9553	0.9553	0.8893	0.8969	0.8969	0.9246	0.9252	0.9252
2	0.8368	0.8414	0.8711	0.9346	0.8925	0.9159	0.8830	0.8662	0.8929
3	0.9540	0.9788	0.9745	0.9421	0.9545	0.9463	0.9480	0.9665	0.9602
4	0.9900	0.9836	0.9808	0.9490	0.9522	0.9777	0.9691	0.9676	0.9793
5	0.8898	0.8769	0.9244	0.9046	0.9461	0.9129	0.8971	0.9102	0.9186
6	0.9244	0.9481	0.9091	0.8980	0.8939	0.9388	0.9110	0.9202	0.9237
7	0.8304	0.8214	0.8585	0.9490	0.9388	0.9286	0.8857	0.8762	0.8922

Table 12: Logistic Regression class-wise performance across runs

Class	Precision			Recall			F1-score		
	Run1	Run2	Run3	Run1	Run2	Run3	Run1	Run2	Run3
1	0.9915	0.9874	0.9916	0.8893	0.9008	0.9046	0.9376	0.9421	0.9461
2	0.8595	0.8664	0.8800	0.9720	0.9393	0.9252	0.9123	0.9013	0.9021
3	0.9590	0.9873	0.9518	0.9669	0.9669	0.9793	0.9630	0.9770	0.9654
4	0.9806	0.9841	0.9689	0.9682	0.9841	0.9936	0.9744	0.9841	0.9811
5	0.9594	0.9375	0.9714	0.9793	0.9959	0.9876	0.9692	0.9658	0.9794
6	0.9595	0.9833	0.9630	0.9673	0.9592	0.9551	0.9634	0.9711	0.9590
7	0.9457	0.9495	0.9785	0.8878	0.9592	0.9286	0.9158	0.9543	0.9529

Table 13: Random Forest class-wise performance across runs

Class	Precision			Recall			F1-score		
	Run1	Run2	Run3	Run1	Run2	Run3	Run1	Run2	Run3
1	0.9283	0.9252	0.9407	0.8893	0.8969	0.9084	0.9084	0.9109	0.9243
2	0.8272	0.8522	0.8485	0.9393	0.9159	0.9159	0.8797	0.8829	0.8809
3	0.9506	0.9625	0.9547	0.9545	0.9545	0.9587	0.9526	0.9585	0.9567
4	0.9773	0.9776	0.9748	0.9586	0.9745	0.9841	0.9678	0.9761	0.9794
5	0.8415	0.8417	0.8745	0.8589	0.9046	0.8963	0.8501	0.8720	0.8852
6	0.8803	0.8904	0.8941	0.8408	0.8286	0.8612	0.8601	0.8584	0.8773
7	0.9011	0.9239	0.9326	0.8367	0.8673	0.8469	0.8677	0.8947	0.8877

Table 14: Linear SVC class-wise performance across runs

Class	Precision			Recall			F1-score		
	Run1	Run2	Run3	Run1	Run2	Run3	Run1	Run2	Run3
1	0.9668	0.9597	0.9639	0.8893	0.9084	0.9160	0.9264	0.9333	0.9393
2	0.8755	0.8874	0.8889	0.9533	0.9206	0.9346	0.9128	0.9037	0.9112
3	0.9834	0.9876	0.9876	0.9793	0.9835	0.9835	0.9814	0.9855	0.9855
4	1.0000	0.9936	0.9937	0.9841	0.9936	1.0000	0.9920	0.9936	0.9968
5	0.9917	0.9959	1.0000	0.9917	0.9959	1.0000	0.9917	0.9959	1.0000
6	0.9683	0.9760	0.9919	0.9959	0.9959	1.0000	0.9819	0.9859	0.9959
7	0.9697	0.9500	0.9794	0.9796	0.9694	0.9694	0.9746	0.9596	0.9744

Table 15: SVC with polynomial kernel, class-wise performance across runs